

Lab 6: Clustering Techniques

Objective

To implement and compare unsupervised clustering algorithms — **K-Means** and **Hierarchical Clustering** — and evaluate their performance using visualization and silhouette analysis.

Theory

Clustering is an unsupervised learning task that groups data points into clusters based on similarity. Unlike classification, the data are unlabeled, and the goal is to discover inherent structure.

Common Clustering Algorithms

- **K-Means Clustering:** Partitions data into k clusters by minimizing within-cluster variance.
- **Hierarchical Clustering:** Builds a hierarchy of clusters through successive merging (agglomerative) or splitting (divisive).

Evaluation Metric:

- **Silhouette Score:** Measures how similar a point is to its own cluster compared to other clusters. Ranges from -1 (incorrect clustering) to $+1$ (well clustered).

Major Functions Used

- `KMeans(n_clusters, random_state)` — from `sklearn.cluster`
- `AgglomerativeClustering()` — for hierarchical clustering
- `silhouette_score()` — from `sklearn.metrics`
- `linkage(), dendrogram()` — from `scipy.cluster.hierarchy`

Procedure

1. Import Required Libraries

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.datasets import make_blobs
6 from sklearn.cluster import KMeans, AgglomerativeClustering
7 from sklearn.metrics import silhouette_score
8 from scipy.cluster.hierarchy import linkage, dendrogram
```

2. Generate Synthetic Data

```
1 # Create a 2D dataset with 3 clusters
2 X, y_true = make_blobs(n_samples=200, centers=3, cluster_std=0.70,
3     random_state=42)
4 plt.scatter(X[:, 0], X[:, 1], s=50)
5 plt.title("Synthetic Dataset (3 Clusters)")
6 plt.show()
```

3. Apply K-Means Clustering

```
1 kmeans = KMeans(n_clusters=3, random_state=0)
2 labels_kmeans = kmeans.fit_predict(X)
3
4 # Plot results
5 plt.scatter(X[:, 0], X[:, 1], c=labels_kmeans, cmap='viridis', s=50)
6 plt.scatter(kmeans.cluster_centers_[:, 0],
7     kmeans.cluster_centers_[:, 1],
8     c='red', marker='X', s=200, label='Centroids')
9 plt.title("K-Means Clustering")
10 plt.legend()
11 plt.show()
12 print("Silhouette Score (K-Means):", silhouette_score(X,
13     labels_kmeans))
```

Explanation: The K-Means algorithm iteratively updates centroids until cluster assignments stabilize. The silhouette score evaluates clustering quality — the higher, the better.

4. Apply Hierarchical Clustering

```
1 # Perform agglomerative hierarchical clustering
2 hc = AgglomerativeClustering(n_clusters=3, linkage='ward')
3 labels_hc = hc.fit_predict(X)
4
```

```
5 plt.scatter(X[:, 0], X[:, 1], c=labels_hc, cmap='plasma', s=50)
6 plt.title("Hierarchical Clustering (Agglomerative)")
7 plt.show()
8
9 print("Silhouette Score (Hierarchical):", silhouette_score(X,
    labels_hc))
```

Explanation: Agglomerative clustering starts with individual data points and merges them iteratively based on distance, forming a tree-like hierarchy.

5. Dendrogram Visualization

```
1 linked = linkage(X, method='ward')
2 plt.figure(figsize=(8, 5))
3 dendrogram(linked, truncate_mode='lastp', p=12, leaf_rotation=45)
4 plt.title("Dendrogram (Ward Linkage)")
5 plt.xlabel("Cluster Size")
6 plt.ylabel("Distance")
7 plt.show()
```

Observation: The dendrogram helps visualize merging distances and decide the optimal number of clusters.

6. Compare Both Methods

```
1 score_km = silhouette_score(X, labels_kmeans)
2 score_hc = silhouette_score(X, labels_hc)
3
4 print(f"K-Means Silhouette Score: {score_km:.3f}")
5 print(f"Hierarchical Silhouette Score: {score_hc:.3f}")
6
7 if score_km > score_hc:
8     print("K-Means performs better on this dataset.")
9 else:
10    print("Hierarchical clustering performs better.")
```

Interpretation: In general, K-Means performs better for spherical clusters, while hierarchical clustering may capture nested or irregular structures.

7. Full Example Script

```
1 from sklearn.datasets import make_blobs
2 from sklearn.cluster import KMeans, AgglomerativeClustering
3 from sklearn.metrics import silhouette_score
4 from scipy.cluster.hierarchy import linkage, dendrogram
5 import matplotlib.pyplot as plt
6
7 # Data generation
8 X, _ = make_blobs(n_samples=200, centers=3, random_state=42)
9
```

```
10 # K-Means
11 kmeans = KMeans(n_clusters=3, random_state=42)
12 labels_kmeans = kmeans.fit_predict(X)
13
14 # Hierarchical
15 hc = AgglomerativeClustering(n_clusters=3)
16 labels_hc = hc.fit_predict(X)
17
18 # Evaluation
19 print("Silhouette (K-Means):", silhouette_score(X, labels_kmeans))
20 print("Silhouette (Hierarchical):", silhouette_score(X, labels_hc))
21
22 # Visualization
23 plt.subplot(1,2,1)
24 plt.scatter(X[:,0], X[:,1], c=labels_kmeans, cmap='viridis')
25 plt.title("K-Means Clusters")
26
27 plt.subplot(1,2,2)
28 plt.scatter(X[:,0], X[:,1], c=labels_hc, cmap='plasma')
29 plt.title("Hierarchical Clusters")
30 plt.show()
```

Listing 5: K-Means and Hierarchical Clustering Example

Discussion

Both clustering techniques attempt to group similar data, but their mechanisms differ:

- **K-Means:** Efficient, works well for large datasets with spherical clusters.
- **Hierarchical:** Provides visual insights through dendrograms but is computationally expensive for large datasets.

The silhouette score provides a reliable measure for comparing clustering quality.

Exercises

1. Generate datasets with different shapes and densities using `make_blobs()` and `make_moons()`.
2. Apply both K-Means and Hierarchical clustering and compare silhouette scores.
3. Vary the number of clusters ($k = 2$ to $k = 6$) and observe changes in cluster separation.
4. Visualize dendrograms for multiple linkage methods (`ward`, `average`, `complete`).
5. Prepare a brief report comparing both methods.

Outcome

Students will:

- Understand and apply unsupervised clustering algorithms.
- Evaluate clustering results using silhouette scores and visualizations.
- Learn to interpret dendrograms and identify optimal cluster numbers.